



Self-Service DAGs: Event-Driven Design for GitHub Actions and Airflow at Lyft



Ken Obata - Lyft, Senior Software Engineer

About me



Hi ! I'm Ken

I'm a Senior Software Engineer at Lyft and I'm from marketplace driver organization. I build product data platform to support dynamic ride pricing.

Fun fact

- I became a new dad this year. Stretching myself with my career and taking care a 6-month old boy.

Contact

- [linkedin.com/in/ken-obata](https://www.linkedin.com/in/ken-obata)

Today's Agenda

Self-Service DAGs: Event-Driven Design
for GitHub Actions and Airflow at Lyft

Intro, Business Background, Problems	3 min
--------------------------------------	-------

Design pattern	12 min
----------------	--------

What broke in production	5 min
--------------------------	-------

Takeaways and Q&A	5 min
-------------------	-------



1. Intro and Business Background

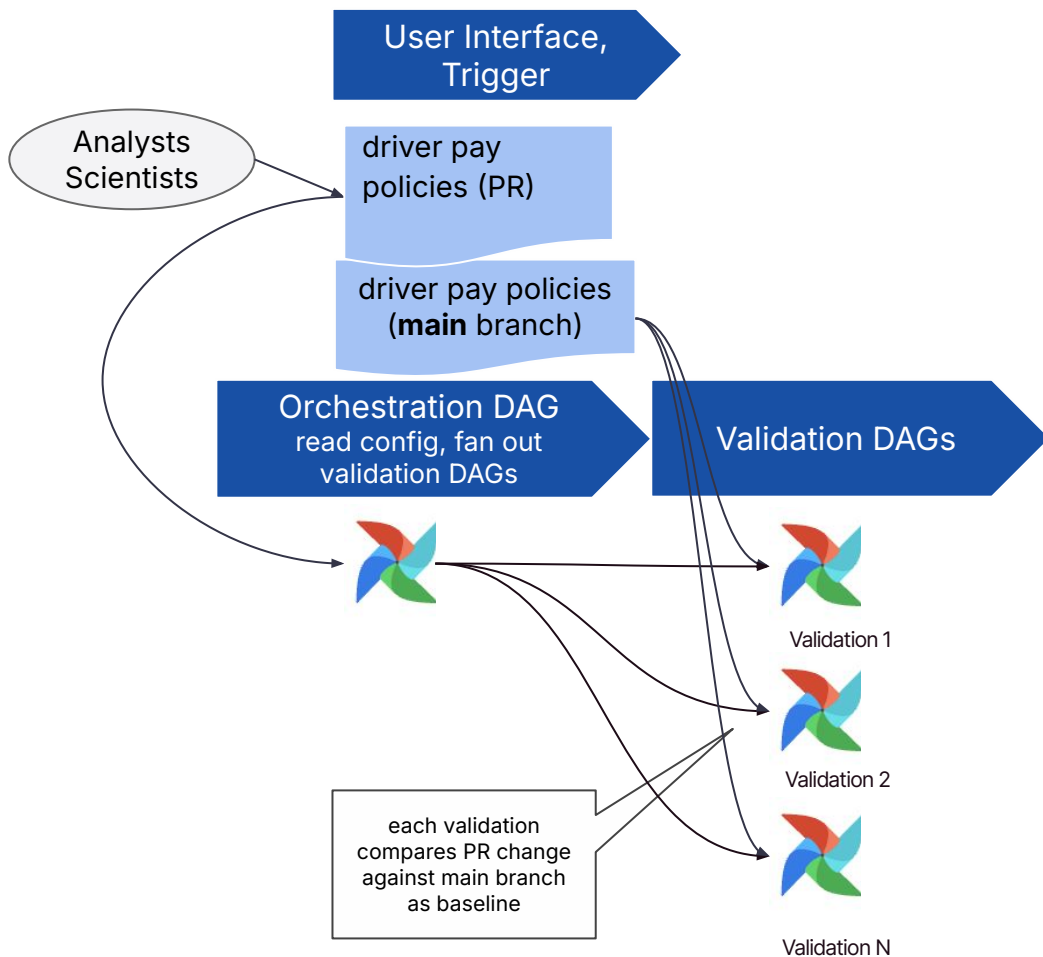
Business Background

- Our Mission is serve and connect.
- At Lyft, driver pay policies define how much drivers should get paid for each ride based on different factors such as distance, duration, ride types etc.
 - ref: <https://help.lyft.com/hc/en-us/driver/articles/115013080008>
- These pay policies are securely IT controlled in github repositories and get updated by Analysts and Scientists.



Our DAG topology

- **Driver pay policies** are controlled by Github.
- **Upstream** DAG acts as Orchestration layer.
- Multiple **downstream** validation DAGs are triggered by orchestration DAG dynamically based on this config as dag conf input.
- Different config input **dynamically** changes how many validation DAGs to run.



Product Requirements and Challenge

Product requirement

- **1.SOX control**
 - config changes need to be **validated** by data pipelines orchestrated by Airflow because these pay-policy changes go straight to what drivers earn.
 - need to **block** PRs until validated.
- **2.config is dynamic**:which validations to run, how many depends on user input.
- **3.Automate** end to end.
 - No data engineer in the loop because Science and analytics owns logic and timing.
 - Many users submit a PR but only a few data engineers. + team timezone diff.

⇒ How can we make DAGs self-service with IT controlled?



Design Pattern

Design Pattern

Legends

1

check ID

Trigger (block)

Process (block)

Complete (unblock)

Users
(Sci/Analyst)



PR

Hit API

/merge

PR status
(UI)

commit xyz

1

CI-check/O

check for orchestrator

1

CI-check

CI-check

2

CI-check/O

3

CI-check/O

4

CI-check/O

2

CI-check

3

CI-check

4

CI-check

Orchestration
DAG



process files &
Trigger DAG
Run (Dynamic
task mapping)

POST

- context ID is unique
per commit ID & dag
run.
- context contains
Astro URL.

POST

check 1

--conf GH context ID
& commit SHA. All
DAGs communicate
through these
context.

completed

Execution
(validation)
DAG

Number of DAGs
depends on user
config input.
Only orchestration
DAG knows.



check 2

check 3

check 4



check 2

check 3

check 4

Design | Why we designed this way: bi-directional, event driven

Why Orchestration DAG?

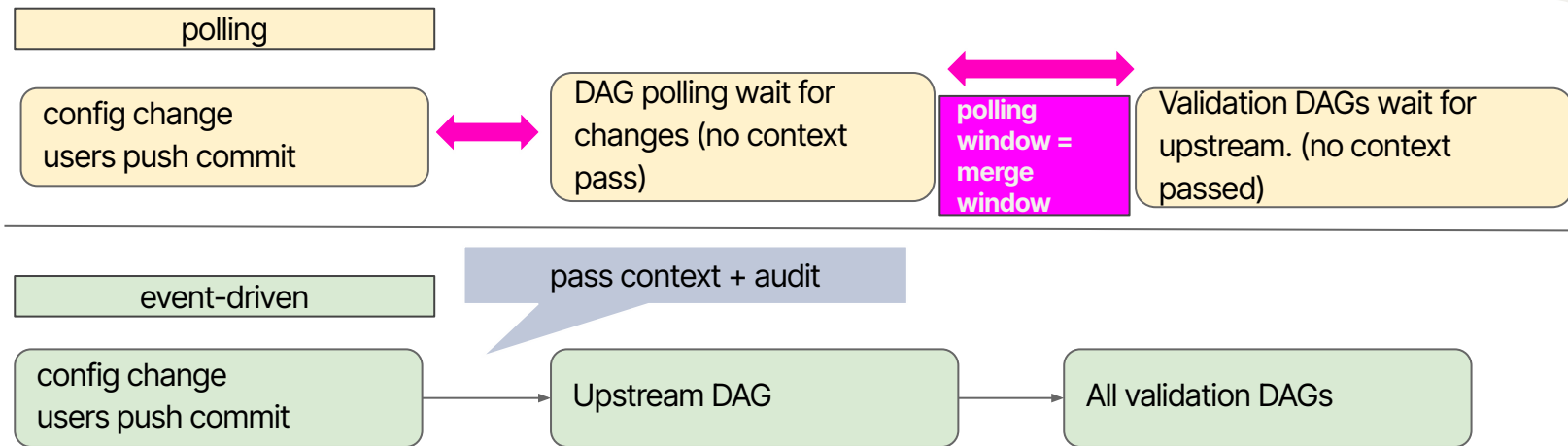
Because business logic (fan-out N validation) lives in Airflow, not GH actions.
[Separation of responsibility]
GH is UI.

Why GH integration & bi-directional?

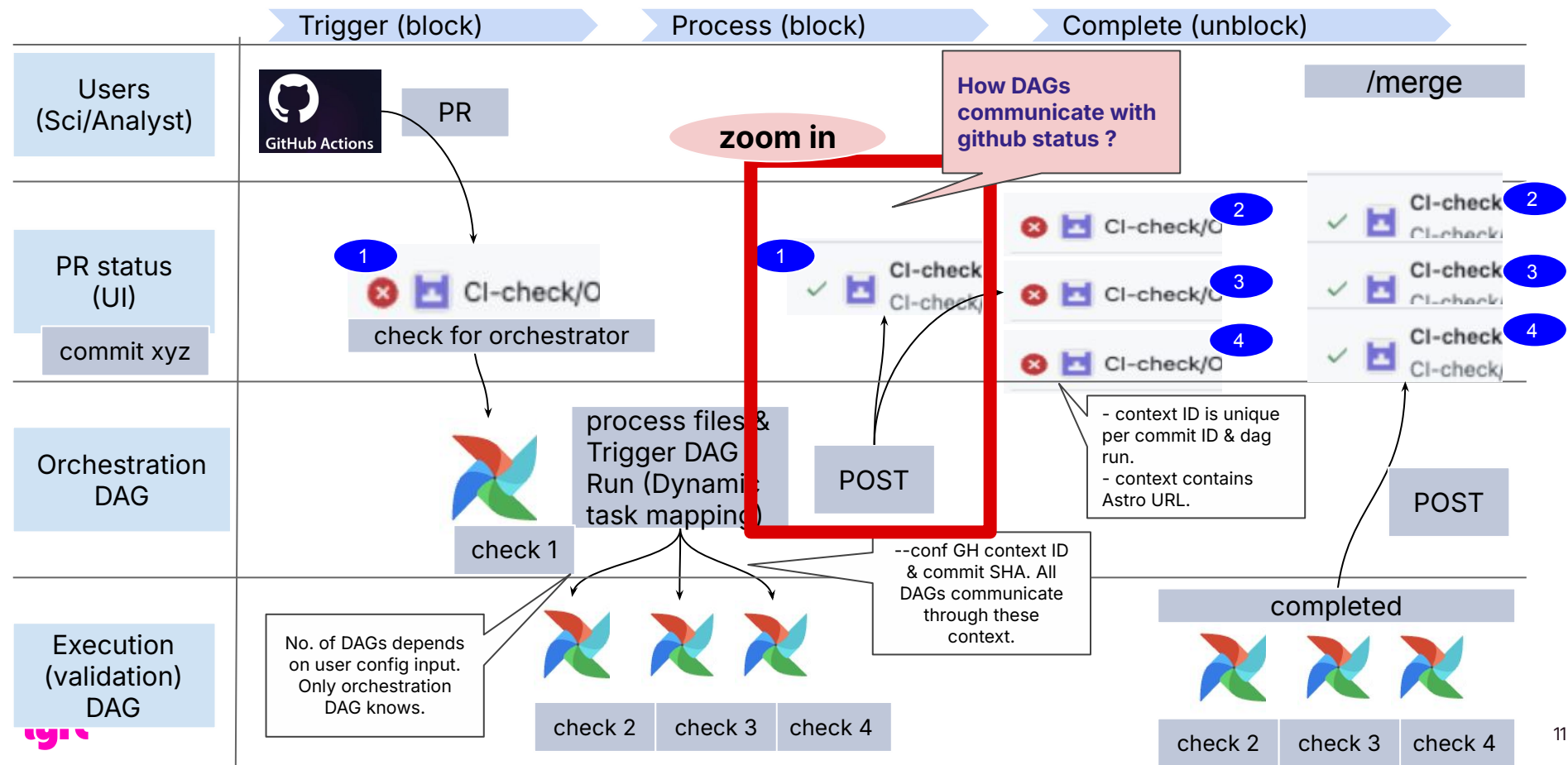
(GH $\Rightarrow$ SOX) without GH integration, it created a merge window.
(bi-directional $\Rightarrow$ separation of responsibility)
GH actions doesn't track validation DAGs

Why event-driven?

Both polling and push can block PRs. **(engineering)** X-DAG context passing (commit sha, PR check ID) + SOX auditability.
(operational) schedule dags don't fit because it's on-demand



Design Pattern | (part 1 of 3: github status)



How DAGs communicate with github status

- All DAGs communicate via github **check context ID** & **commit SHA** to know which specific context DAGs are responsible for.
- Notice that the commit SHA is in the API endpoint. The **PR status is bound to that specific commit**. Hold onto that because this is a critical piece in "What broke in production" section.

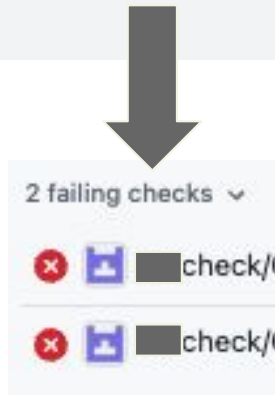
[Status update step: simplified code]

```
Python
payload = {
    "state": status,
    "context": context,
    "description": context,
    "target_url": target_url,
}

url =
f"https://api.github.com/repos/{organization}/{repo_name}/statuses/{commit_hash}"
headers = _create_api_headers()

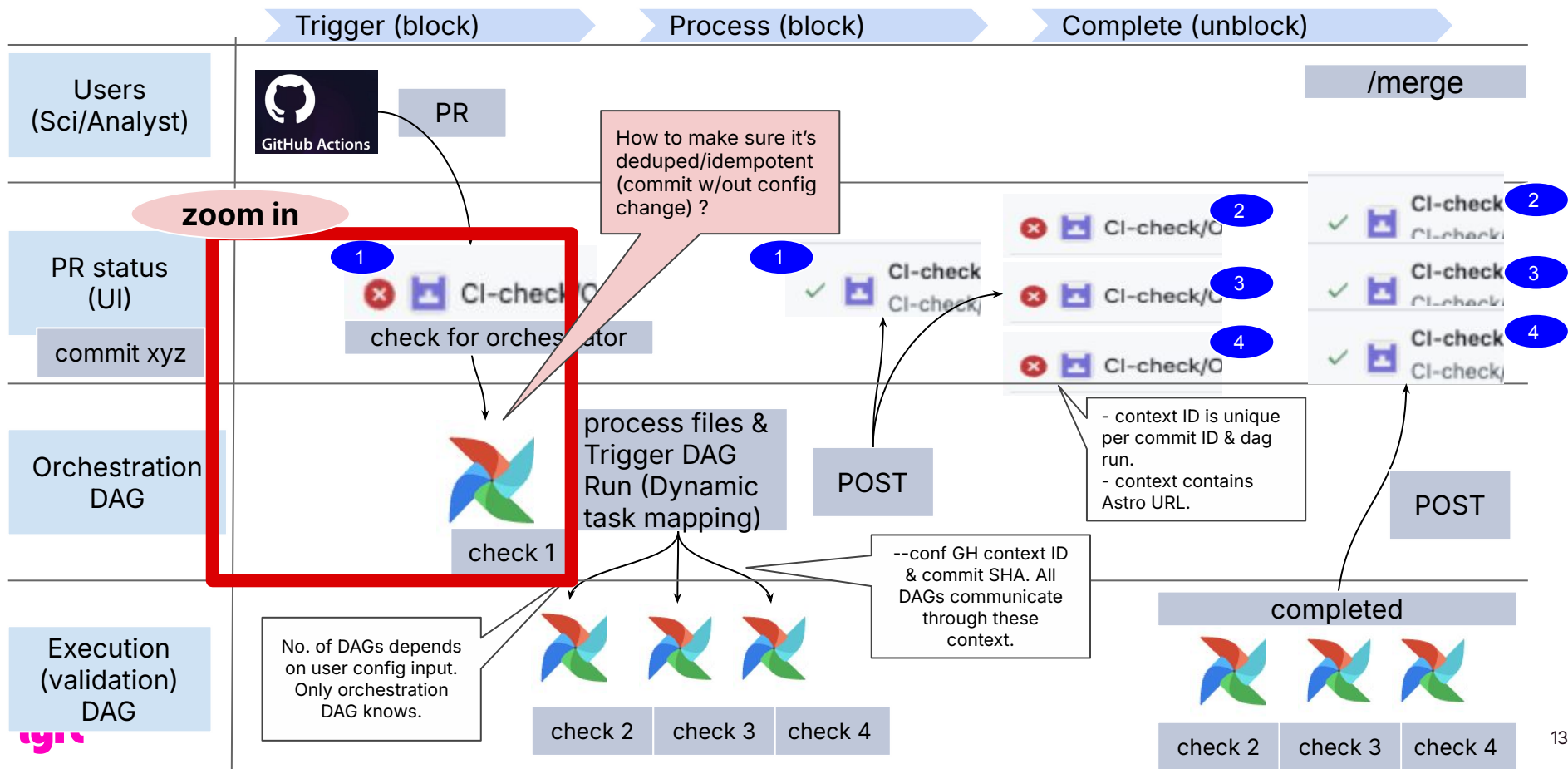
try:
    response = requests.post(
        url,
        json=payload,
        headers=headers,
        timeout=60,
    )
```

commit sha is part of the endpoint



Each check has context ID. Tied to commit sha.

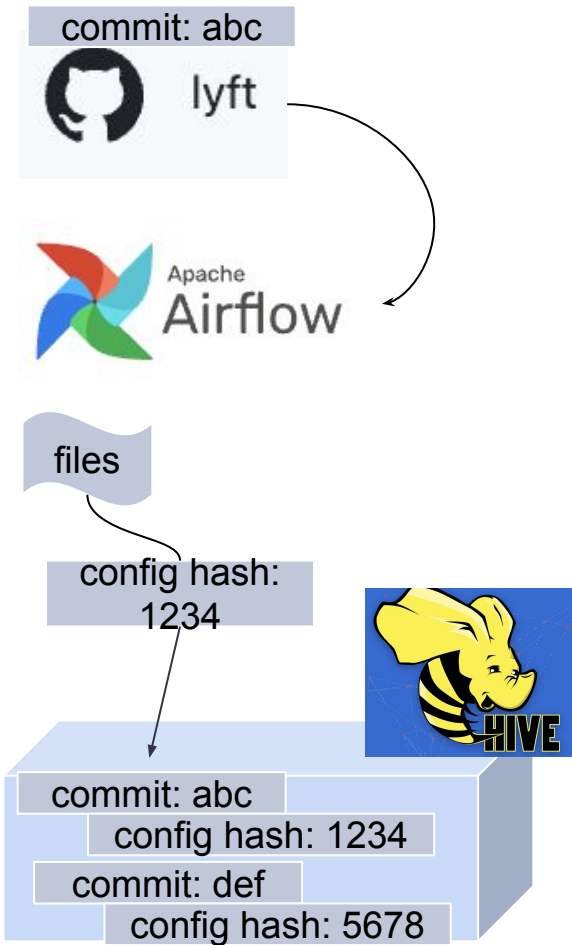
Design Pattern | (part 2 of 3: Idempotency)



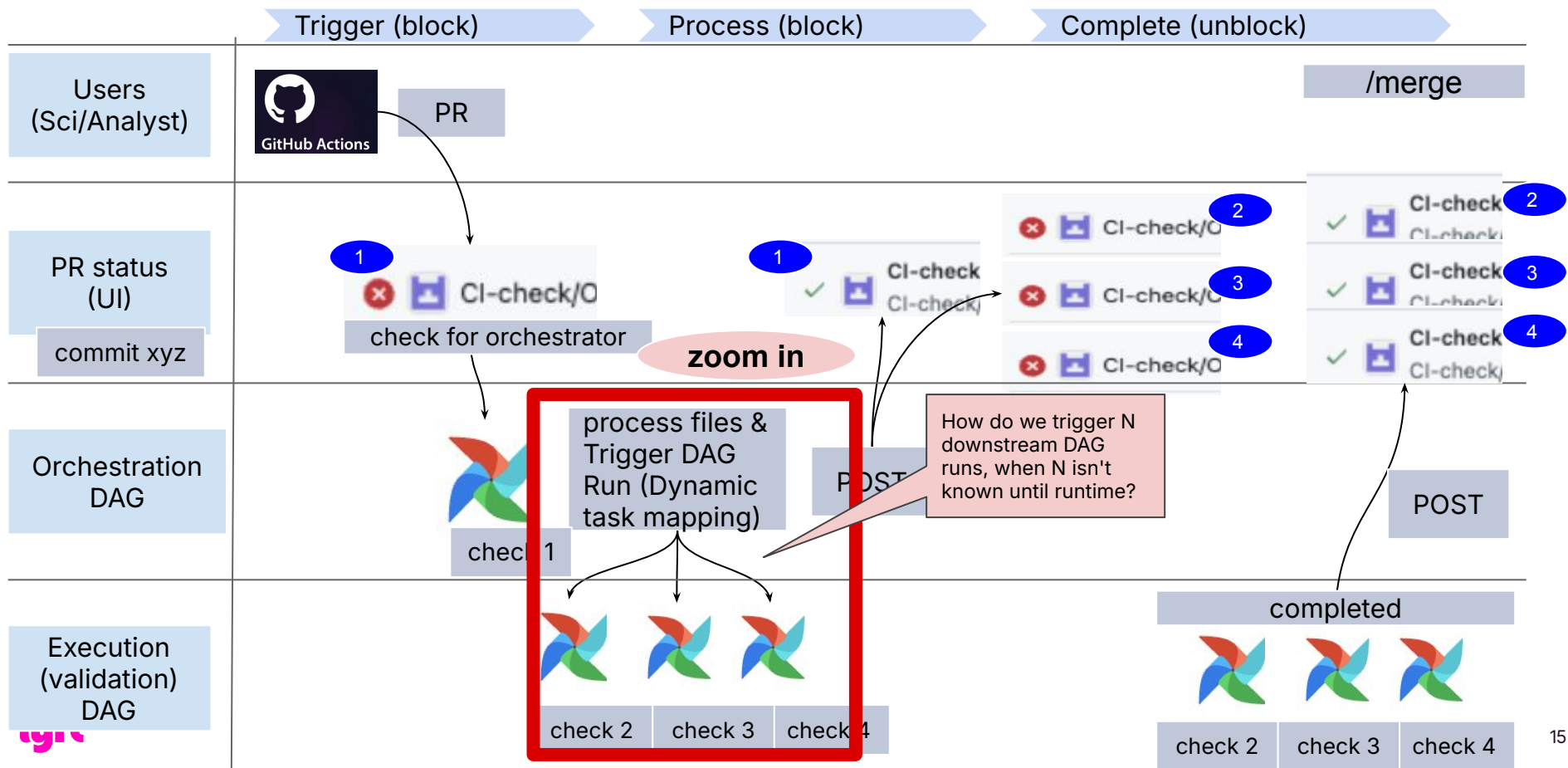
Design | Idempotency processing

Context: users make multiple commits in a PR. But not all commits are about config changes. (e.g. doc, renaming, etc.)

- **Concept:** Idempotency/deduping is on processing, not triggering.
- **Challenge:** how to tell if configs have changed?
 - create a hash from config contents. Store into config history table and track if the hash is changed
- **Why** external table instead of building logic in GH action?:
 - GH Actions has no durable store.
- **Why** external table instead of task state store?
 - task state store is run scoped, this is for cross dag.
- **Why** not asset state store (it's cross-DAG and durable)?
 - external query engines (e.g. Spark) need access to this table.
- Result: duplicate or racing triggers are safe by construction. A re-trigger finds the same hash and skips.



Design Pattern | (part 3 of 3: triggering N downstream dynamically)



Design | Trigger Dagrun Operator vs Asset Event Trigger

- **Context** : the orchestration DAG used TriggerDagRunOperator with dynamic task mapping to trigger N downstream dag runs dynamically based on user input.
- AFv3 introduced the asset-based scheduling which is also event-driven.
 - asset trigger does not support dynamic task mapping yet ([ref](#) v3.2).
- In fact, they don't compete with each other.

Tool	Concept	Use case
Event Asset Trigger	decoupling lineage	- Fan-out via decoupling: upstream announces freshness without knowing its consumers. - not much control on passing context to downstream.
TriggerDagRunOperator (w/ dynamic task mapping)	coupling lineage	- lineage is tightly coupled, passing context, run_id control, and options to wait for downstream completion. - dynamic task mapping of N runs with distinct conf

What broke in production

What broke in production | edge cases

Context: Needed to trigger the DAG for every commit because users can change their experiment config every commit. Triggering DAGs every commit is not a problem because we have idempotent mechanism.

edge case - rebased PR

- **problem 1. [User experience]** check status from original commits are gone because **check statuses are tied to a commit hash**. rebase $\Rightarrow$ a new commit hash $\Rightarrow$ previous check statuses are orphaned.
- **problem 2. [data consistency]** when DAGs are triggered, original commit hash is passed to downstream DAGs. By the time downstream DAGs POST commit status, **commit hash can be outdated**.



What broke in production | edge cases

More edge case: multiple rebases

- Reminder - DAGs communicate via context ID from original commit. When rebase happens, original context IDs are gone from github. We want to copy original context IDs to the rebased commit.
 - Another reason why a check status is updated from Airflow instead of GH actions.
- GH actions supports last commit but it might not be original commit and check statuses.

Failed Solution

- Using github action to get PR checks from last commit before rebase push and copying over to new commit
 - **Last commit != original commit.** There can be multiple rebase.

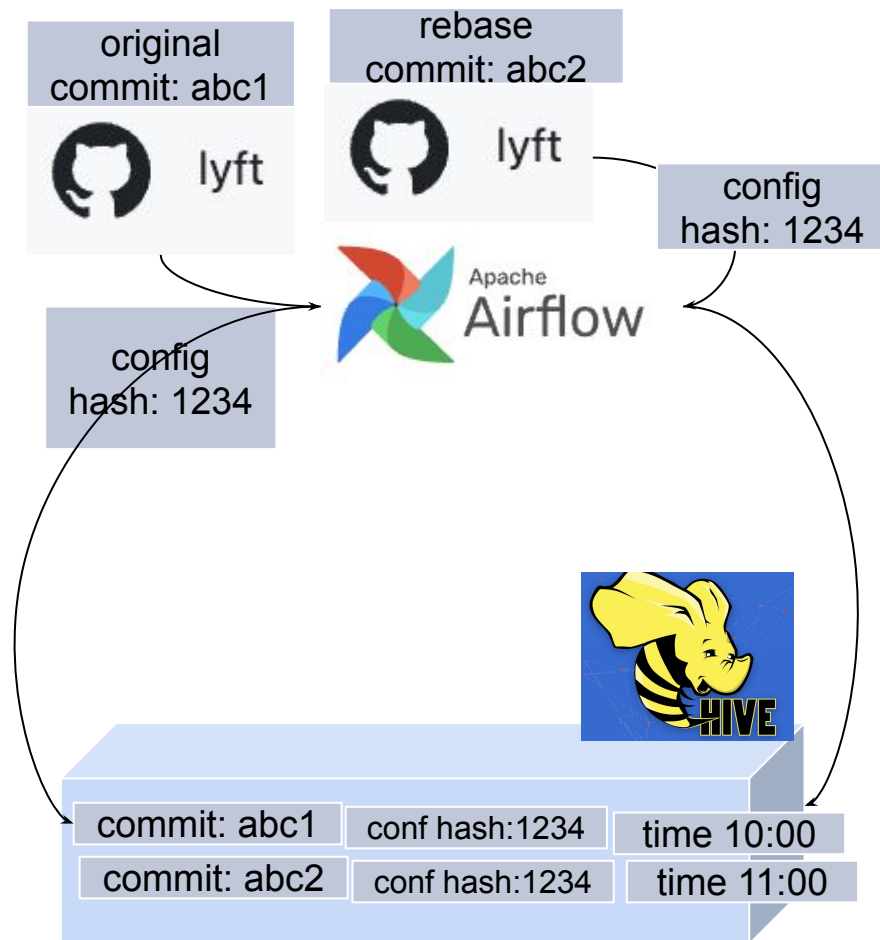
```
Python
head = payload.get("head")      # New commit (after push)
before = payload.get("before")  # Old commit (before push)
```

What broke in production | Working Solution

Idea: DAGs copy context ID and check status from the original commit to the latest commit.

How: Have a history table with a tuple (config_hash, github commit, timestamp)

- 1. rebase has same config_hash. Fetch original commit from the config_hash.
- 2. Using original commit_sha, Airflow runs GET API to retrieve original commit's PR checks and their status.
- 3. Airflow runs POST API against current commit hash with original context_id (check ID) and their statuses.



5. Leanings and Q&As

5. Takeaways

- 1.Event-driven compounds when the communication is bidirectional
 - Inbound (GH → Airflow) makes validation self-service and IT controlled.
 - Outbound (Airflow → GH) the PR page becomes the UI.
- 2.Assets decouple and announce. TriggerDagRunOperator couples and parameterizes.
 - We need both, at different layers. Asset triggers announce freshness; they don't carry a payload. The moment upstream needs to pass contexts (e.g. commit_sha, context_id), then TriggerDagRunOperator fits more.
- 3.GitHub Actions integration is more than just hitting the API endpoint
 - Edge case of rebase and data consistency are challenging. Having a metadata history DB will achieve **idempotency processing** and data consistency among multiple DAGs.
 - **This is a heavy lift.** Worth it when the UI/config is critical enough to need audited control, the users aren't data engineers, and self-service is the requirement.
 - **But the cost is front-loaded:** once the framework exists, extending it to new DAGs is cheap.



Questions?



Contact

[linkedin.com/in/ken-obata](https://www.linkedin.com/in/ken-obata)